Fachhochschule Nordwestschweiz
Hochschule für Technik

Batchnorm

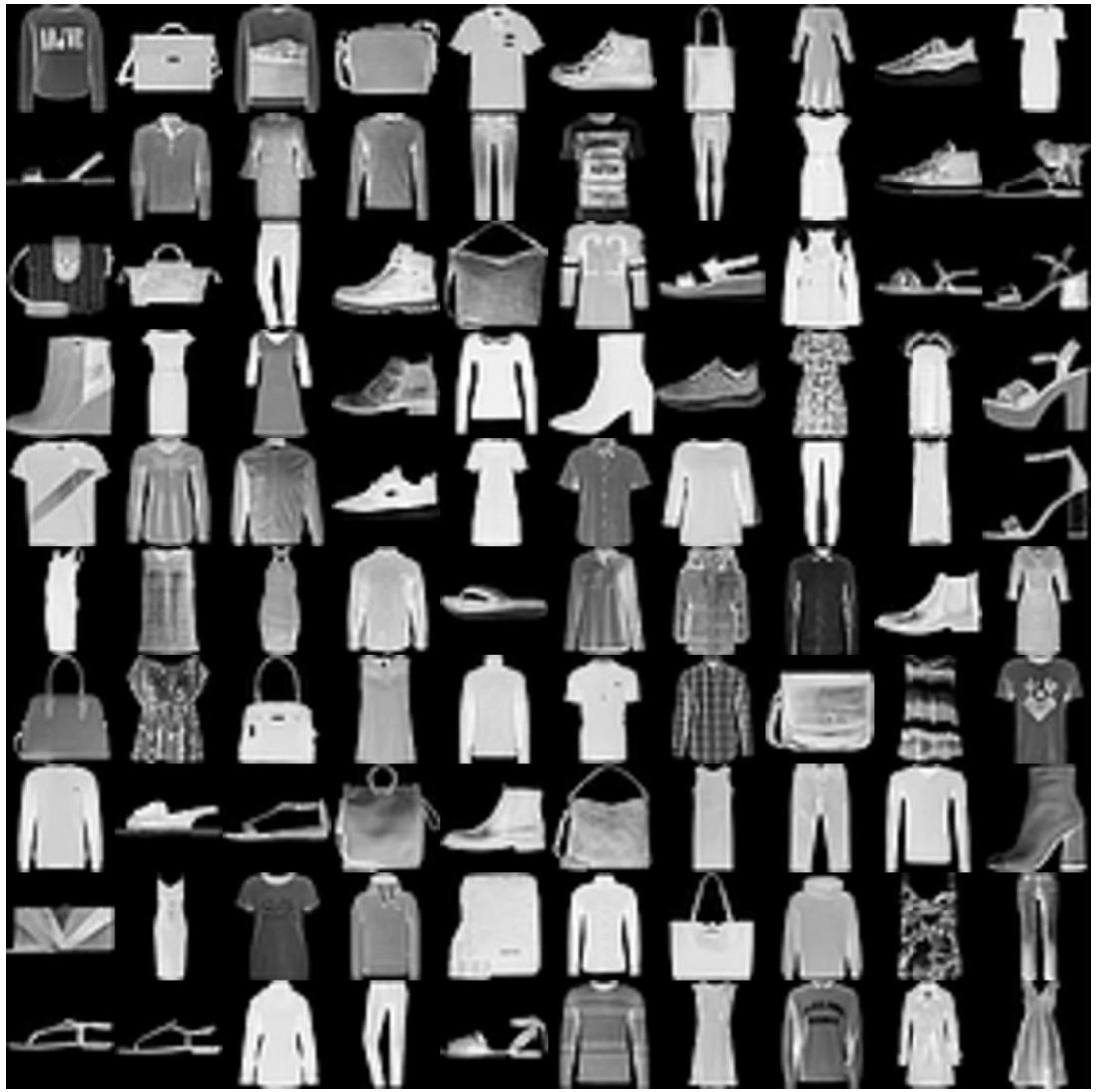
Deep Learning

HS 2025

Martin Melchior

FashionMNIST

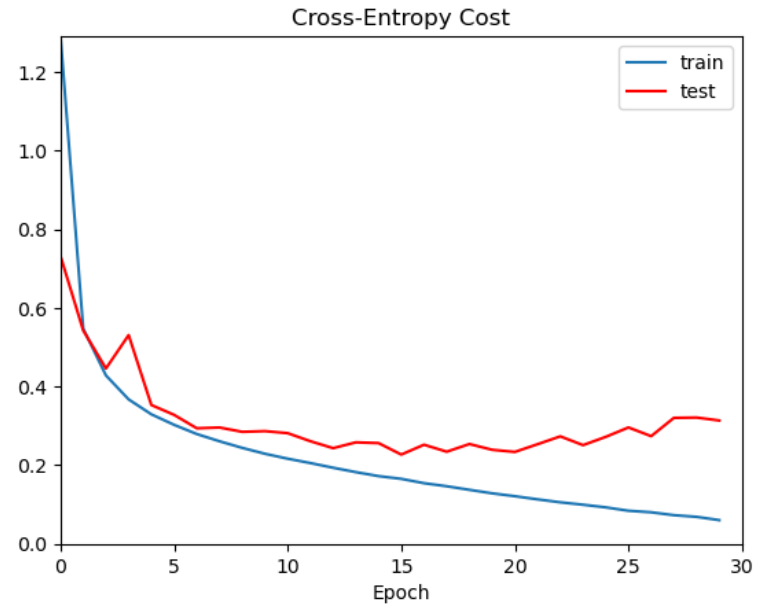
Same structure as MNIST,
but different conten:
clothes, boots, bags

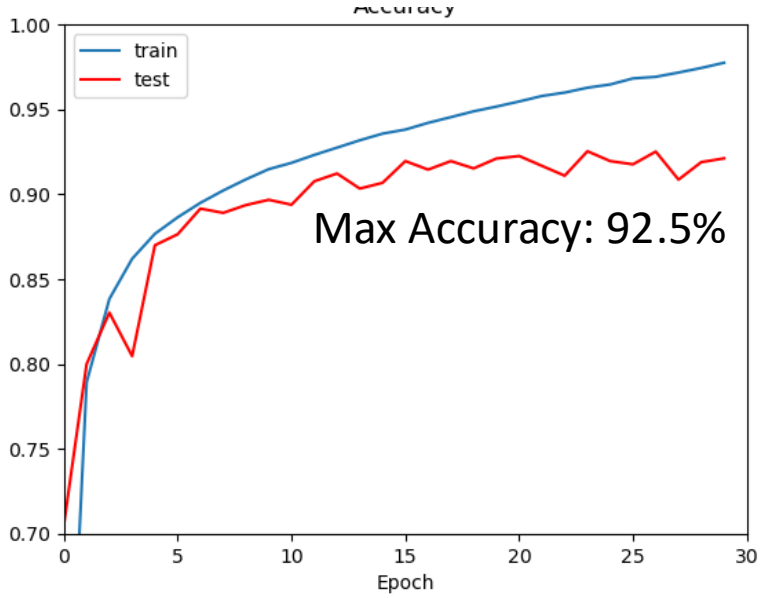


Model

```python
class CNN(torch.nn.Module):

    def __init__(self, nhidden1=256, nhidden2=64, noutput=10):
        super(CNN, self).__init__()
        self.layers = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1, bias=True),
            torch.nn.ReLU(),
            torch.nn.Conv2d(in_channels=32, out_channels=32, kernel_size=3, padding=1, bias=True),
            torch.nn.ReLU(),
            torch.nn.MaxPool2d(kernel_size=3, stride=2, padding=1),
            torch.nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1, bias=True),
            torch.nn.ReLU(),
            torch.nn.Conv2d(in_channels=64, out_channels=64, kernel_size=3, padding=1, bias=True),
            torch.nn.ReLU(),
            torch.nn.MaxPool2d(kernel_size=3, stride=2, padding=1),
            torch.nn.Flatten(),
            torch.nn.Linear(7*7*64, nhidden1),
            torch.nn.ReLU(),
            torch.nn.Linear(nhidden1, nhidden2),
            torch.nn.ReLU(),
            torch.nn.Linear(nhidden2, noutput)
        )

    def forward(self, x):
        z = self.layers(x)
        return z
```

Model with 3 CNN, 2 linear and one output layer
Slightly overfitting after ~20 epochs



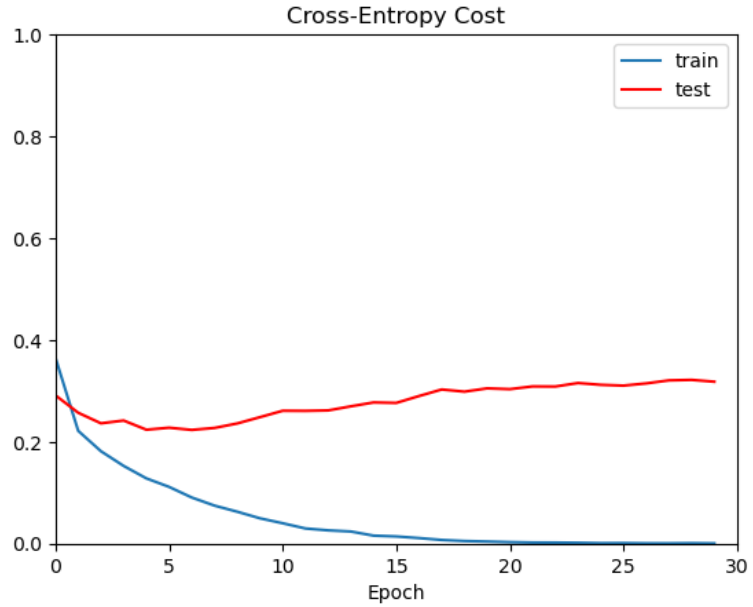


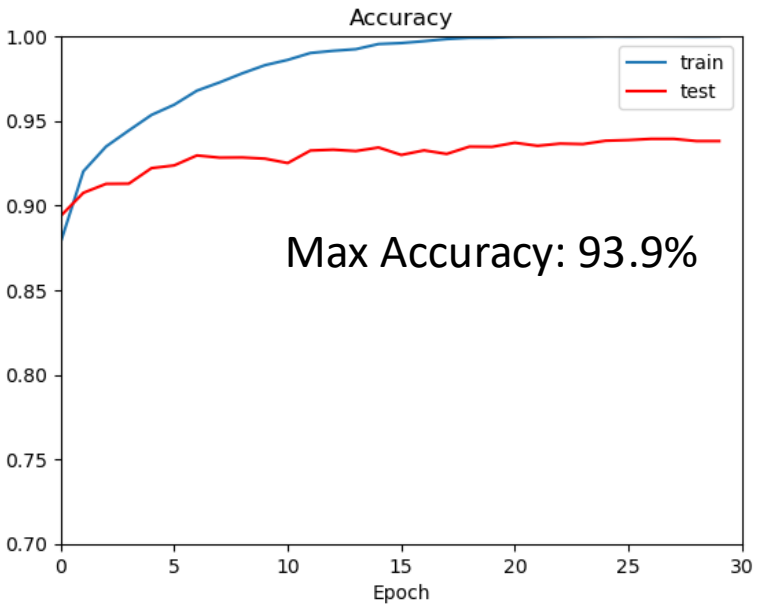
Model with BatchNorm

```
class CNNWithBatchnorm(torch.nn.Module):

    def __init__(self, nhidden1=240, nhidden2=80, dropout_rate=0.7, noutput=10):
        super(CNNWithBatchnorm, self).__init__()
        self.layers = torch.nn.Sequential(
            torch.nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1, bias=True),
            torch.nn.BatchNorm2d(32),
            torch.nn.ReLU(),
            torch.nn.Conv2d(in_channels=32, out_channels=32, kernel_size=3, padding=1, bias=True),
            torch.nn.BatchNorm2d(32),
            torch.nn.ReLU(),
            torch.nn.MaxPool2d(kernel_size=3, stride=2, padding=1),
            torch.nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1, bias=True),
            torch.nn.BatchNorm2d(64),
            torch.nn.ReLU(),
            torch.nn.Conv2d(in_channels=64, out_channels=64, kernel_size=3, padding=1, bias=True),
            torch.nn.BatchNorm2d(64),
            torch.nn.ReLU(),
            torch.nn.MaxPool2d(kernel_size=3, stride=2, padding=1),
            torch.nn.Flatten(),
            torch.nn.Linear(7*7*64, nhidden1),
            torch.nn.BatchNorm1d(nhidden1),
            torch.nn.ReLU(),
            torch.nn.Linear(nhidden1, nhidden2),
            torch.nn.BatchNorm1d(nhidden2),
            torch.nn.ReLU(),
            torch.nn.Linear(nhidden2, noutput)
        )

    def forward(self, x):
        z = self.layers(x)
        return z
```

`BatchNorm2d` for conv layers

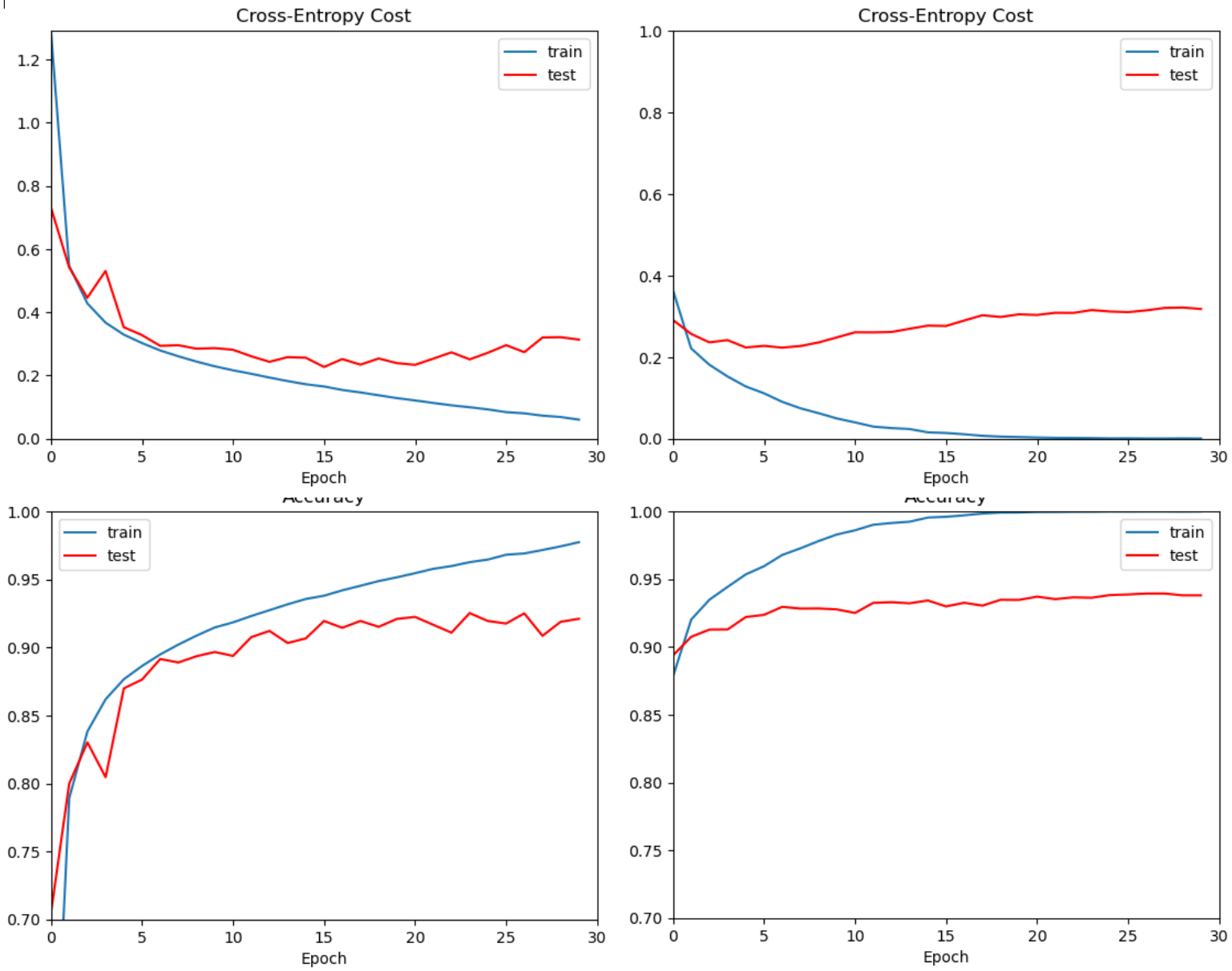
`BatchNorm1d` for linear layers





Max Accuracy: 93.9%

with batchnorm



Training
- faster,
- more robust
- resulting in slightly better performance (accuracy).

BatchNorm

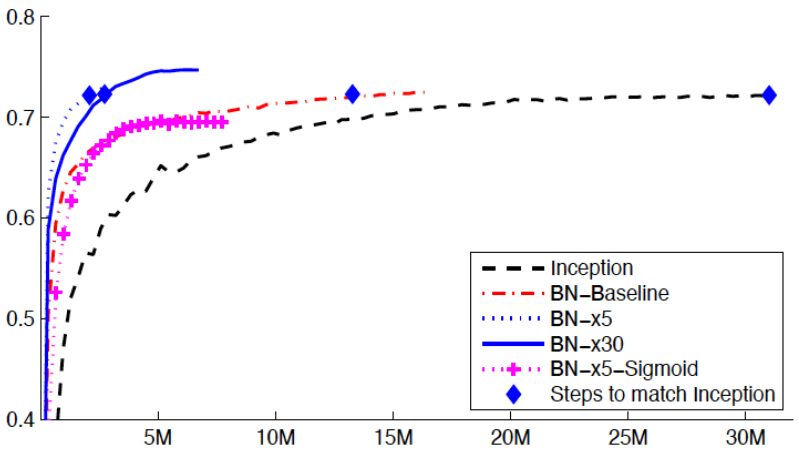


Figure 2: *Single crop validation accuracy of Inception and its batch-normalized variants, vs. the number of training steps.*

from the original paper:
"Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift"
by Sergey Ioffe, Christian Szegedy

Model	Steps to 72.2%	Max accuracy
Inception	$31.0 \cdot 10^6$	72.2%
BN-Baseline	$13.3 \cdot 10^6$	72.7%
BN-x5	$2.1 \cdot 10^6$	73.0%
BN-x30	$2.7 \cdot 10^6$	74.8%
BN-x5-Sigmoid		69.8%

Figure 3: *For Inception and the batch-normalized variants, the number of training steps required to reach the maximum accuracy of Inception (72.2%), and the maximum accuracy achieved by the network.*

First applied to Inception Network (GoogleLeNet), taken as "baseline" (black line).

Observed great speed-up in the training:

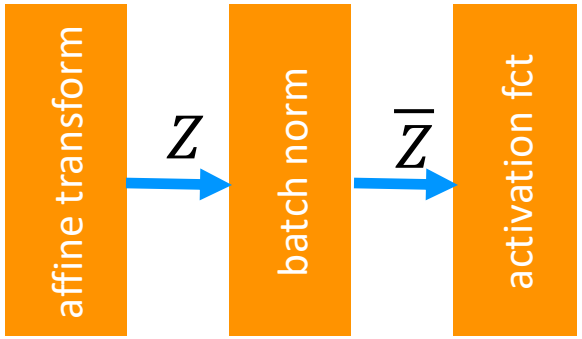
- Time to reach accuracy of baseline (72.2%) much reduced: factor ~2.3
- Allowed for increased learning rates (blue lines: 5x or 30x larger): factor ~14.8
  Increased accuracy!
- Even works with ReLU replaced by Sigmoid.

Batch Normalisation

Idea introduced by S. Ioffe and C. Szegedy(1):

- Normalise the logits Z of a layer with an estimate of the mean and stdev of the current mini-batch (per feature).
- Adjust the normalised logits $\overline{Z}$ with a trainable scale and shift parameter (γ, β).
- Apply the activation function.

Applicable to any layer in a network.



$$
\begin{aligned}
Z &= W \cdot A^{prev} + \cancel{b} \\
\tilde{Z} &= Z - \frac{1}{b}\sum_j Z^{(j)} \\
\hat{Z} &= \frac{\tilde{Z}}{\sqrt{\epsilon + \frac{1}{b}\sum_j \tilde{Z}^2}} \\
\overline{Z} &= \gamma \hat{Z} + \beta
\end{aligned}
$$

component-wise operations

component-wise learned parameters

bias term b can be ignored since it can be compensated with β

(1) S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift” (2015).

BatchNorm at Production Time

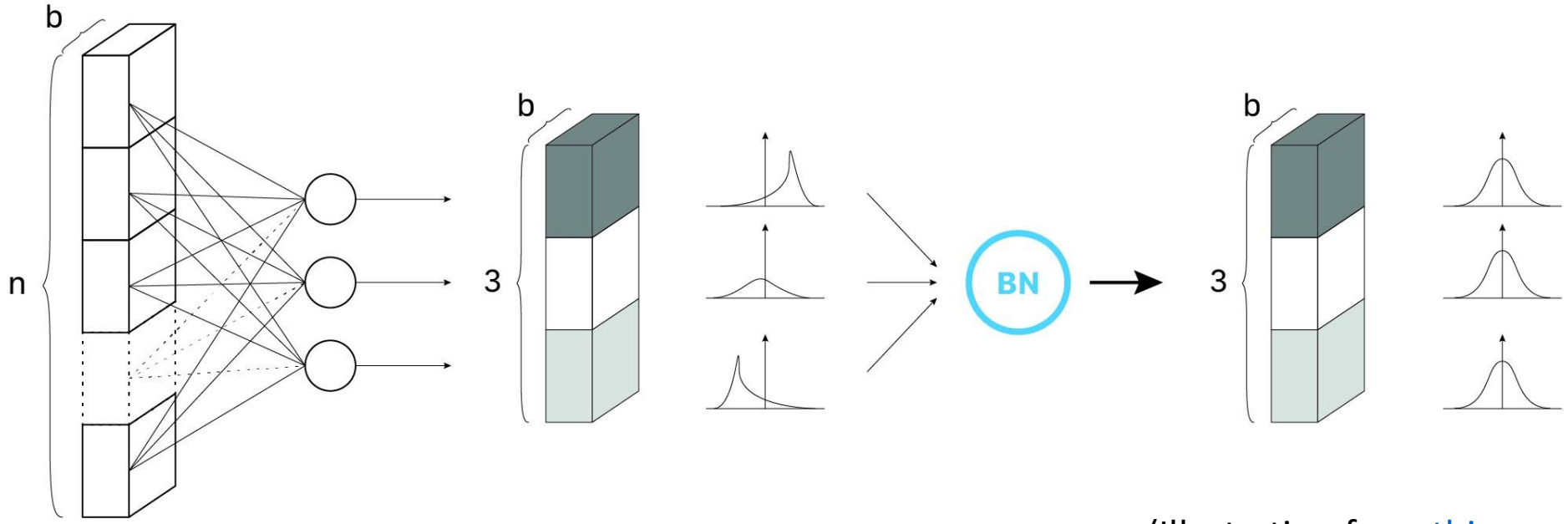
At test time or in production we don't have batches!

- Mean and stdev computed per mini-batch (per feature and layer) can be used to compute mean and stdev (per feature and layer) for the population.
- Use exponentially weighted averages $\hat{\mu}^{(t)}, \hat{\sigma}^{(t)}$ with a decay rate ν, estimated at training time (for each feature and layer) by
  - $\hat{\mu}^{(t+1)} = (1-\nu)\hat{\mu}^{(t)} + \nu\mu_{\text{batch}}^{(t)}$
  - $\hat{\sigma}^{(t+1)} = (1-\nu)\hat{\sigma}^{(t)} + \nu\sigma_{\text{batch}}^{(t)}$
- Use these quantities at inference time for normalisation.
- Different behavior of BatchNorm at train and valid time!

```
for epoch in range(nepochs):
    # parameter update
    model.train()
    for batch, (X, Y) in enumerate(tr
        pred = model(X)
        loss = cost_ce(pred, Y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    # evaluation
    model.eval()
    ...
```

Effect of Batch Norm

Makes distribution of input values to each unit in a neural network distributed similarly to a standard normal distribution (during training) - before adjusting it with the learnable scale and shift parameter.

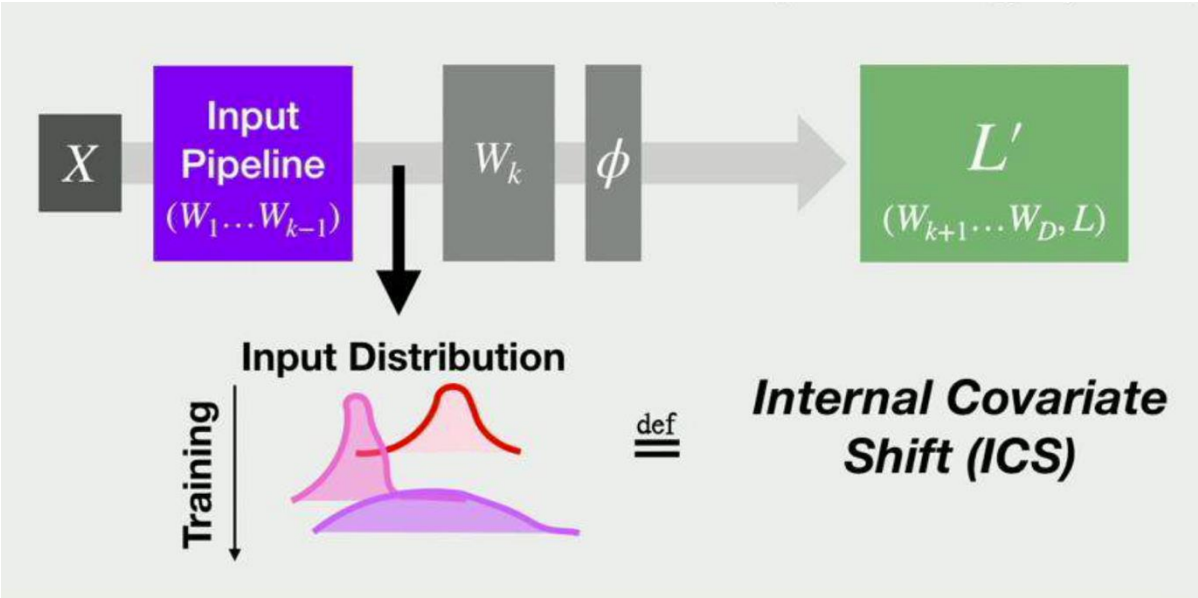


Network layer with 3 units, batch size b.

(Illustration from [this source])

Why does BatchNorm Work?

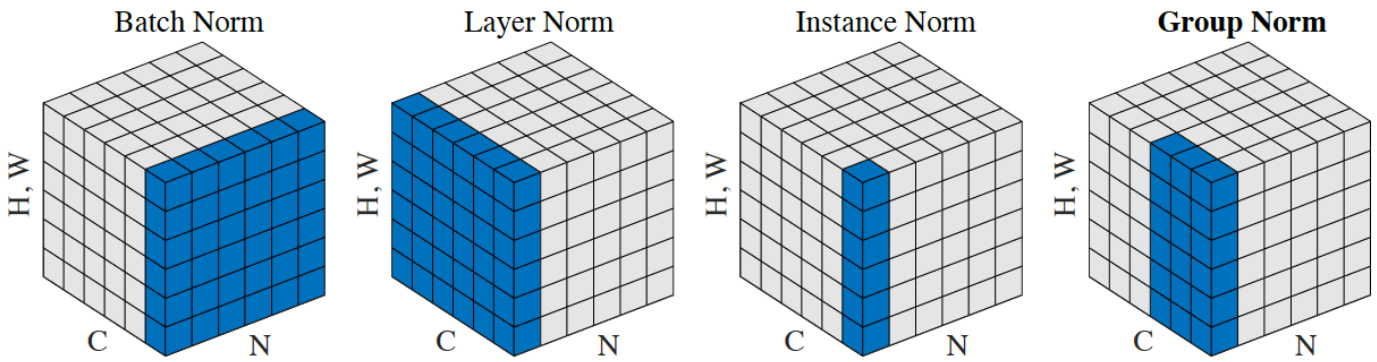
- Original idea of Ioffe and Szegedy: Alleviate internal covariate shift problem.
- This intuition is however controversial: S.Santukar et al (NIPS 2018) indicate that the hypothesis might be wrong.
- They rather state that the effect of BatchNorm is that it makes
  - the updates in the different layers more independent,
  - the cost surface smoother (gradients change less rapidly),
  - hence makes training more robust.



A given layer (W_k) sees during training varying distributions of input values from previous layer ("Input Pipeline") because the weights in the previous layers get updated which affects the distribution of their activations. Subsequent layers (L') need to continually adapt.

Modifications, Generalisations for CNNs

- Batch Norm: over samples and (H,W), per channel
- Layer Norm: over channels and (H,W), per sample.
  - Gives slightly worse improvements than Batch Norm, but can be performed per sample, i.e. similar behaviour in train and test.
- Instance Norm: over (H,W), per channel, per sample



J. L. Ba, J. R. Kiros, and G. E. Hinton.
Layer normalization. CoRR,
abs/1607.06450, 2016.

Y. Wu and K. He.
Group normalization. CoRR,
abs/1803.08494, 2018.

BatchNorm and Dropout

BatchNorm also has a regularising effect; possibly (if there is sufficient training data) it is sufficient and you can skip using other regularization methods (e.g. Dropout).

Debate on the order to put Dropout and BatchNorm layers:

- BatchNorm before / after non-linearity: In the original paper before non-linearity. Forum discussions tend to give preference putting BatchNorm after non-linearity.
- BatchNorm before / after Dropout: In this blog post by figs_and_nuts, the scheme

CONV/FC -> ReLu(or other activation) -> Dropout -> BatchNorm -> CONV/FC

is suggested with the following reasoning:
Dropout meant to block information from certain neurons to make sure neurons do not co-adapt.
BatchNorm should come after dropout because otherwise information from all neurons is passed normalisation statistics.

BatchNorm: Final Remarks

- Caution:
  - Batch size should not be too small for batch norm to be applicable
  - Different handling at training and test time may be delicate
    -> LayerNorm gained popularity.

- Modifies Backprop. Why?